

GUIA TECNICA PASO A PASO

Fundamentos de Arquitectura LLM

Sesion 2 — Implementacion LLM: Local, Cloud y Escalabilidad

Ing. Andrés Felipe Rojas Parra

BSG Institute

Tema	Tecnologias	Caso de Uso
Tema 1: Local	Python 3.12, Ollama, FastAPI	Analisis de Riesgo Crediticio
Tema 2: Azure	Azure AI Foundry, GPT-4o, FastAPI	Deteccion de Fraude Bancario
Tema 3: Escala	Docker, Kubernetes, HPA, RabbitMQ	Arquitectura de Produccion

Docente: AI/LLM Senior Solution Architect | Duracion: 3 horas

TEMA 1 — Analisis de Riesgo Crediticio con Ollama (Windows Local)

Aprenderemos a construir una API de analisis de credito que usa un LLM 100% local, sin enviar datos de clientes a la nube. Caso de uso: banco colombiano que necesita cumplir la Ley 1581 de proteccion de datos.

1.1 Prerequisitos e Instalacion en Windows

Paso 1: Instalar Python 3.12

Abre PowerShell como Administrador y ejecuta:

```
winget install Python.Python.3.12  
python --version # Debe mostrar Python 3.12.x
```

Tip: Si winget no esta disponible, descarga Python desde <https://python.org/downloads>

Paso 2: Instalar Ollama (motor LLM local)

Ollama es el runtime que permite ejecutar modelos de lenguaje en tu laptop:

```
winget install Ollama.Ollama
```

Alternativa: Descargar el instalador desde <https://ollama.ai/download/windows>

```
ollama --version # Verificar instalacion
```

Paso 3: Descargar el modelo LLM

Descargamos Llama 3.2:3b — un modelo de 2GB que corre bien en laptops sin GPU:

```
ollama pull llama3.2:3b  
# La descarga tarda 2-5 minutos segun conexion  
ollama list # Ver modelos disponibles
```

Nota: 1 token en Ollama local cuesta \$0.00 — sin limites, sin factura, sin internet.

Paso 4: Clonar el repositorio y configurar entorno

```
git clone https://github.com/<tu-usuario>/llm-sesion2.git  
cd llm-sesion2  
  
# Crear entorno virtual  
python -m venv .venv  
.venv\Scripts\activate  
  
# Instalar dependencias  
pip install -r requirements.txt
```

```
# Configurar variables de entorno
copy .env.example .env
```

El archivo .env por defecto ya esta configurado para uso local, no necesitas cambiar nada para el Tema 1.

Paso 5: Ejecutar la API

```
# Opcion 1: usando Makefile (recomendado)
make run-local

# Opcion 2: directamente
uvicorn examples.local_analyzer.main:app --reload --port 8001

# Abrir en browser
http://localhost:8001/docs
```

1.2 Arquitectura del Sistema

El flujo de datos en el sistema local es:

- 1. Cliente HTTP (browser/curl) envia solicitud de credito a FastAPI
- 2. FastAPI valida el schema con Pydantic v2 (tipos seguros)
- 3. CreditAnalyzer calcula DTI, score y construye el prompt
- 4. Ollama (Llama 3.2:3b) genera el analisis narrativo en local
- 5. La respuesta JSON con recomendacion se retorna al cliente

Componente	Tecnologia	Puerto	Proposito
API REST	FastAPI + Uvicorn	8001	Endpoint de solicitudes
Validacion	Pydantic v2	—	Schemas y tipos seguros
Logica	CreditAnalyzer	—	DTI, scoring cuantitativo
LLM Local	Ollama (Llama 3.2:3b)	11434	Analisis narrativo
Retry	Tenacity	—	3 reintentos con backoff

1.3 Probar los Endpoints

Health Check

```
curl http://localhost:8001/health
```

Respuesta esperada:

```
{"status": "healthy", "ollama_connected": true, "model_loaded": true, "uptime_seconds": 42.1}
```

Demo rapido (sin construir JSON)

```
curl http://localhost:8001/credit/demo
```

El endpoint /credit/demo ejecuta un caso preconfigurado y retorna el analisis completo.

Analisis completo via Swagger UI

Navega a <http://localhost:8001/docs> y usa el endpoint POST /credit/analyze con este JSON:

```
{
  "applicant": {
    "applicant_id": "APL-2024-001",
    "age": 34,
    "employment_status": "empleado",
    "monthly_income_usd": 1200.0,
    "years_employed": 5.0,
    "existing_debts_usd": 250.0,
    "credit_history_years": 8,
    "previous_defaults": 0,
    "city": "Bogota",
    "credit_score": 680
  },
  "credit_type": "personal",
  "requested_amount_usd": 8000.0,
  "requested_term_months": 36,
  "purpose": "Remodelacion de vivienda propia"
}
```

Respuesta esperada (ejemplo):

```
{
  "request_id": "REQ-A4F2B8C1",
  "risk_score": 74,
  "risk_level": "bajo",
  "recommendation": "APROBAR",
  "indicators": {
    "debt_to_income_ratio": 28.4,
    "estimated_monthly_payment_usd": 283.50
  },
  "ai_analysis": "El solicitante presenta DTI de 28.4% dentro del umbral...",
  "processing_time_ms": 4823.2,
  "privacy_note": "Análisis procesado 100% local"
}
```

CRITICO: Ningun dato del solicitante sale de tu laptop. Total cumplimiento Ley 1581.

TEMA 2 — Deteccion de Fraude con Azure AI Foundry

Construiremos una API de deteccion de fraude que usa GPT-4o desplegado en Azure AI Foundry — la plataforma enterprise de Microsoft para IA.

2.1 Configurar Azure AI Foundry

Paso 1: Crear cuenta Azure

Si no tienes cuenta Azure, crea una gratuita en <https://portal.azure.com> (incluye \$200 de credito).

Paso 2: Crear un proyecto en Azure AI Foundry

6. Ve a <https://ai.azure.com> y haz click en 'Create new project'
7. Nombre del proyecto: 'Ilm-latam-fraud'
8. Region: 'East US 2' (o Brazil South si disponible)
9. Haz click en 'Create'

Paso 3: Desplegar GPT-4o

10. En el proyecto, ve a 'Deployments' > 'Deploy model'
11. Selecciona 'gpt-4o' del catalogo de modelos
12. Nombre del deployment: 'gpt-4o' (este sera tu AZURE_AI_MODEL)
13. Tokens per minute: 100K (suficiente para demos)

Paso 4: Obtener credenciales

En tu proyecto de Azure AI Foundry:

- Ve a 'Project settings' > 'Keys and endpoints'
- Copia el 'Endpoint' — formato: <https://<proyecto>.services.ai.azure.com/models>
- Copia la 'Primary key' (API Key)

Paso 5: Configurar .env

```
# Editar el archivo .env con tus credenciales
AZURE_AI_ENDPOINT=https://tu-proyecto.services.ai.azure.com/models
AZURE_AI_KEY=tu-api-key-de-azure
AZURE_AI_MODEL=gpt-4o
AZURE_AI_API_VERSION=2024-12-01-preview
```

Paso 6: Ejecutar la API de fraude

```
make run-azure
# O directamente:
uvicorn examples.azure_foundry.main:app --reload --port 8002
```

```
# Swagger UI: http://localhost:8002/docs
```

2.2 Probar Deteccion de Fraude

Demo con transaccion sospechosa

```
curl http://localhost:8002/fraud/demo
```

Este endpoint ejecuta una transaccion de \$4,850 USD realizada a las 2:47 AM en Miami por un usuario colombiano cuyo promedio es \$85 USD. Debe retornar BLOQUEAR.

Analisis personalizado

```
curl -X POST http://localhost:8002/fraud/analyze \
-H "Content-Type: application/json" \
-d '{"transaction_id": "TXN-TEST-001",
  "amount_usd": 1250.0,
  "card": {"last_four": "4521", "card_type": "credit", "issuing_country": "CO"},
  "merchant": {"name": "Amazon", "category": "online_retail", "country": "US",
  "city": "Seattle"},
  "user_profile": {"user_id": "USR-789", "avg_transaction_usd": 95.0,
    "typical_countries": ["CO","US"], "transactions_last_24h": 2,
    "account_age_days": 730}}'
```

Senal de Fraude	Descripcion	Impacto en Score
Monto inusual	Tx > 5x el promedio del usuario	Alto
Pais inusual	Comerciante en pais no habitual	Alto
Horario nocturno	Transaccion entre 1am y 5am UTC	Medio
Alta frecuencia	Mas de 10 tx en ultimas 24h	Alto
Dispositivo desconocido	Nuevo fingerprint o proxy	Medio
Categoria riesgo	Casino, crypto, gift cards	Muy alto

TEMA 3 — Patrones Arquitectonicos y Escalabilidad

Aprendemos a desplegar nuestras APIs en contenedores y a escalarlas automaticamente en Kubernetes.

3.1 Docker y Docker Compose

Levantar el stack completo

```
# Requiere Docker Desktop instalado
make docker-up

# Verificar contenedores corriendo
docker compose -f docker/docker-compose.yml ps
```

El docker-compose.yml levanta 6 servicios:

Servicio	Puerto	Descripcion
api-local	8001	API FastAPI + Ollama (Tema 1)
api-azure	8002	API FastAPI + Azure (Tema 2)
ollama	11434	Motor LLM local
redis	6379	Cache de respuestas
prometheus	9090	Metricas de la API
grafana	3000	Dashboard de monitoreo (admin/llm2024)

3.2 Kubernetes — Escalabilidad automatica

Desplegar en Kubernetes local (minikube)

```
# Instalar minikube (si no lo tienes)
winget install Kubernetes.minikube
minikube start --memory=4096 --cpus=2

# Crear namespace
kubectl create namespace llm-latam

# Desplegar
make k8s-deploy

# Verificar
kubectl get pods -n llm-latam
kubectl get hpa -n llm-latam
```

El HPA (Horizontal Pod Autoscaler)

El HPA monitorea CPU y memoria, y agrega o elimina pods automaticamente:

- Si CPU > 70% en promedio: agrega un pod
- Si CPU < 50% por 5 minutos: elimina un pod
- Minimo siempre 2 pods (alta disponibilidad)
- Maximo 10 pods (control de costos)

```
# Simular carga para ver el HPA en accion
kubectl run load-test --image=busybox --rm -it --restart=Never -- \
  sh -c 'while true; do wget -q -O- http://llm-api-service/health; done'

# Observar el autoscaling en tiempo real
kubectl get hpa -n llm-latam --watch
```

3.3 Patrones Clave para LLMs

Patron	Descripcion	Cuando Usar
Circuit Breaker	Si el LLM falla 5 veces, deja de llamarlo por 30s	LLM externo (Azure, OpenAI)
Rate Limiter	Maximo N requests/segundo por cliente	APIs publicas
Cache semantico	Cachear respuestas similares (Redis)	Preguntas frecuentes
Async Queue	RabbitMQ para procesar en background	Alto volumen, batch
Retry+Backoff	3 reintentos con espera exponencial	Siempre recomendado
Health Check	/health para K8s liveness/readiness	Produccion

3.4 Comparativa: Local vs Azure AI Foundry

Criterio	Local (Ollama)	Azure AI Foundry
Privacidad	100% local, cero riesgo	DPA requerido
Costo por token	\$0.00	\$0.15-\$10 por 1M tokens
Calidad modelo	Buena (Llama 3.2:3b)	Excelente (GPT-4o)
Latencia	3-8s (sin GPU)	0.8-2s

Escalabilidad	Limitada por hardware	Automatica
SLA	Tu responsabilidad	99.9% Microsoft
Ideal para	Bancos, salud, gobierno	Startups, apps publicas

Entregables del Proyecto — Sesión 2

N	Entregable	Puntos	Criterio de Exito
1	API Local (Tema 1) funcional	30 pts	/credit/analyze retorna analisis completo
2	API Azure o Mock (Tema 2)	25 pts	/fraud/analyze retorna veredicto
3	Docker Compose levanta	20 pts	docker compose up inicia todos los servicios
4	k8s/deployment.yaml con HPA	15 pts	kubectl apply -f k8s/ sin errores
5	README con instrucciones	10 pts	Otro alumno puede replicar en 15 min

FECHA DE ENTREGA: Antes de la Sesión 3. Entregar via Pull Request al repositorio del curso.

Repositorio de referencia: <https://github.com/arojaspa76/Repo-AI-LLM-Solutions-Architect-Project>